

Práctica 2 - Generación de Variables Aleatorias Discretas**Vianka Vanessa Castro Ordoñez – 23201**

Planteamiento del problema

Una tienda de conveniencia vende botellas de agua y, debido al espacio limitado de almacenamiento, su propietario ha decidido abastecerse con **tres botellas al inicio de cada día**. No obstante, ha observado que la cantidad de clientes que solicitan este producto varía diariamente, por lo **que algunos días logra vender todo su inventario**, mientras que **en otros pierde ventas por no contar con suficientes existencias**. Después de analizar el historial de ventas, se determinó que la demanda diaria de botellas puede modelarse mediante la siguiente distribución de probabilidad:

Demanda diaria (botellas)	Probabilidad
0	0.05
1	0.15
2	0.30
3	0.30
4	0.15
5	0.05

El propietario desea conocer si mantener un inventario fijo de tres botellas por día es una estrategia adecuada o si esta política ocasiona pérdidas por falta de producto o por exceso de inventario. Como consultor, usted ha sido contratado para desarrollar una simulación que represente el comportamiento de la demanda durante un período de **15 días consecutivos**. Para ello, deberá implementar un programa que utilice la distribución de probabilidad proporcionada para generar la demanda diaria mediante números aleatorios. Al finalizar la simulación, el programa deberá presentar la información necesaria para evaluar el desempeño de la política de inventario utilizada. Con base en los resultados obtenidos, elabore un breve análisis en el que determine si abastecerse con tres botellas diarias es una decisión adecuada o si sería recomendable modificar esta cantidad.

Entregables

El programa deberá:

- Simular la operación de la tienda durante 15 días.
-

Mostrar, como mínimo, el número aleatorio generado, la demanda simulada y el resultado de la operación de cada día.

- Presentar un resumen con las métricas que considere pertinentes para evaluar la política de inventario.
- Incluir una conclusión fundamentada en los resultados de la simulación.

Demanda	Probabilidad	Acumulada	Intervalo
0	0.05	0.05	$0.00 \leq U < 0.05$
1	0.15	0.20	$0.05 \leq U < 0.20$
2	0.30	0.50	$0.20 \leq U < 0.50$
3	0.30	0.80	$0.50 \leq U < 0.80$
4	0.15	0.95	$0.80 \leq U < 0.95$
5	0.05	1.00	$0.95 \leq U < 1.00$

Para colocar la probabilidad de la demanda se simulará un número aleatorio entre 0 y 1.

Esto se acumula para evaluar la posición de dicho valor dentro del intervalo que le corresponde a la

demanda.

Tenemos 3 posibilidades:

1. Si demanda es 2 :
 - a. 1 sobra
2. Si demanda es 3:
 - a. 0 Sobra
3. Si demanda es 5:
 - a. 2 ventas perdidas

Ventas = mínimo entre demanda y el inventario disponible

Inventario Sobrante = Máx 3 – demanda y 0

Ventas Perdidas = Máx entre demanda – 3 y 0.

```
emanda.py
----- RESUMEN DE LA SIMULACIÓN -----
Demanda total: 34
Ventas totales: 31
Botellas sobrantes: 14
Ventas perdidas: 3
Días con inventario sobrante: 10
Días con inventario exacto: 3
Días con inventario insuficiente: 2
Promedio de demanda diaria: 2.27
```

```
emanda.py
----- RESUMEN DE LA SIMULACIÓN -----
Demanda total: 45
Ventas totales: 34
Botellas sobrantes: 11
Ventas perdidas: 11
Días con inventario sobrante: 6
Días con inventario exacto: 1
Días con inventario insuficiente: 8
Promedio de demanda diaria: 3.0
```

```
----- RESUMEN DE LA SIMULACIÓN -----
Demanda total: 40
Ventas totales: 35
Botellas sobrantes: 10
Ventas perdidas: 5
Días con inventario sobrante: 5
Días con inventario exacto: 5
Días con inventario insuficiente: 5
Promedio de demanda diaria: 2.67
```

Con respecto a estos resultados corri el programa varias veces y se observa que se mantiene un inventario diario de tres botellas está bien ya que el promedio de demanda diaria cae dentro de ese rango. Sin embargo, esto no elimina por completo las ventas perdidas, cuando tomamos en cuenta las 3 posibilidades de demanda las 3 botellas fueron suficientes o incluso también hubo sobrantes. Pero hay otros días en donde la demanda creció y no fue posible dar más botellas.

Considero que sería mejor incrementar la cantidad de botellas diarias a 4 aunque habrían más días con inventario sobrante, tendría menos posibilidades de que pierda una venta durante dos semanas.

Ejercicio 2

Proporcione un algoritmo eficiente para simular el valor de una variable aleatoria X tal que

$$P\{X = 1\} = 0,3, \quad P\{X = 2\} = 0,2, \quad P\{X = 3\} = 0,35, \quad P\{X = 4\} = 0,15.$$

X	Probabilidad	Acumulada	Intervalo
1	0.30	0.30	$0 \leq U < 0.30$
2	0.20	0.50	$0.30 \leq U < 0.50$
3	0.35	0.85	$0.50 \leq U < 0.85$
4	0.15	1.00	$0.85 \leq U < 1.00$

Es casi el mismo principio que el ejercicio anterior ya que vamos colocando por medio de intervalos la probabilidad de que alguno ocurra

```
import random

u = random.random()

if u < 0.30:
    x = 1
elif u < 0.50:
    x = 2
elif u < 0.85:
    x = 3
else:
    x = 4

print("Número aleatorio:", round(u, 4))
print("Valor de X:", x)
```

Ejercicio 3

Una baraja de 100 cartas —numeradas 1,2,...,100— se mezcla y luego se voltean las cartas una a la vez. Se dice que ocurre un “acierto” cuando la carta i es la i -ésima carta en ser volteada, para $i = 1,...,100$. Escriba un programa de simulación para estimar la esperanza y la varianza del número total de aciertos. Ejecute el programa. Encuentre las respuestas exactas y compárelas con sus estimaciones.

The screenshot shows the Visual Studio Code interface with the following components:

- EXPLORER:** Shows the file structure with `ejercicio3.py` selected.
- EDITOR:** Contains the Python code for the simulation.
- TERMINAL:** Shows the output of running the script.

Python Code (ejercicio3.py):

```

11 for i in range(repeticiones):
12     # baraja de cartas
13     cartas = list(range(1, 101))
14
15     # barajear cartas
16     random.shuffle(cartas)
17
18     # contar los aciertos
19     aciertos = 0
20
21     for j in range(len(cartas)):
22         if cartas[j] == j + 1:
23             aciertos += 1
24
25     resultados.append(aciertos)
26
27
28 # Calcular la esperanza
29 esperanza = sum(resultados) / repeticiones
30
31 # Calcular la varianza
32 suma = 0
33 for x in resultados:
34     suma += (x - esperanza) ** 2
35
36 varianza = suma / repeticiones
37
38 print("Esperanza:", round(esperanza, 4))
39 print("Varianza:", round(varianza, 4))

```

Terminal Output:

```

> > >
Días con inventario exacto: 3
Días con inventario insuficiente: 2
Promedio de demanda diaria: 2.27
PS C:\Projects\Compiladores-Actividades\ejercicio1> cd ..
PS C:\Projects\Compiladores-Actividades> py ejercicio3.py
Esperanza: 0.9974
Varianza: 0.9969
PS C:\Projects\Compiladores-Actividades> py ejercicio3.py
Esperanza: 0.9972
Varianza: 0.9974
PS C:\Projects\Compiladores-Actividades>

```

Si X = numero de aciertos

Y entre cada 100 cartas tiene probabilidad $1/100$ de quedar en su posición correcta

Por linealidad de esperanza:

$$E(X) = 100 (1/100) = 1$$

La esperanza al mezclar las 100 cartas esperamos en promedio 1 acierto

Varianza

$$\text{Var}(X)=1$$

Matemáticamente también indica que tanto la simulación como la exacta son muy cercanas. Esto demuestra que la simulación puede llegar a acercarse demasiado al resultado matemático exacto cuando se ejecuta varias veces.

Ejercicio 4

Utilizando un procedimiento eficiente, junto con la secuencia de números aleatorios del texto, genere una secuencia de 25 variables aleatorias de Bernoulli independientes, cada una con parámetro $p = 0,8$. ¿Cuántos números aleatorios fueron necesarios?

Una variable de Bernoulli con $p = 0.8$ toma el valor 1 con probabilidad 0.8 y el valor 0 con probabilidad 0.2. En lugar de gastar un número aleatorio por cada posición, se debe iniciar las 25 posiciones en 1 y generar únicamente las distancias entre los ceros. Estas distancias siguen una distribución geométrica con parámetro 0.2.

1. Crear una lista de 25 unos.
2. Generar U y calcular $Y = \text{int}(\log(U) / \log(0.8)) + 1$.
3. Avanzar Y posiciones y colocar un cero si la posición todavía pertenece a la lista.
4. Repetir hasta que el siguiente salto supere la posición 25.

Se usa la semilla 23201 que permite repetir exactamente la ejecución. La secuencia concreta y la cantidad de números usados cambiarían si se sustituyera por la secuencia de números aleatorios del texto, pero el algoritmo sería el mismo.

```
py ejercicio4.py  
EJERCICIO 4 - BERNOULLI(p = 0.8)  
Secuencia generada: [0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
1, 1, 1, 0, 1, 1]  
Cantidad de unos: 23  
Cantidad de ceros: 2  
Numeros aleatorios utilizados: 3  
PS C:\Projects\Compiladores-Actividades>
```

en esta ejecución se necesitaron tres números aleatorios: dos produjeron posiciones válidas para los ceros y el tercero produjo el salto final que detuvo el procedimiento. Esto muestra el ahorro frente a utilizar 25 números aleatorios.

Ejercicio 5

Se lanza continuamente un par de dados justos hasta que todos los resultados posibles 2,3,...,12 hayan ocurrido al menos una vez. Desarrolle un estudio de simulación para estimar el número esperado de lanzamientos de dados que se necesitan.

Este problema es una variante del coleccionista de cupones, pero las once sumas no tienen la misma probabilidad. Las sumas centrales aparecen con mayor frecuencia y las sumas 2 y 12, con probabilidad $1/36$ cada una, suelen determinar el tiempo de espera.

5. Lanzar dos dados y guardar la suma obtenida en un conjunto.
6. Continuar hasta que el conjunto contenga todas las sumas de 2 a 12.
7. Repetir el experimento 100,000 veces y promediar los lanzamientos.
8. Comparar la simulación con la esperanza calculada mediante inclusión-exclusión.

```
PS C:\Projects\Compiladores-Actividades> py ejercicio5.py
Simulaciones: 100,000
Promedio estimado: 61.1486
Error estandar: 0.1129
Esperanza teorica: 61.2174
Diferencia absoluta: 0.0688
PS C:\Projects\Compiladores-Actividades>
```

se requieren aproximadamente 61 lanzamientos para observar al menos una vez todas las sumas. La diferencia de 0.0688 respecto de la esperanza teórica es menor que un error estándar, por lo que el resultado simulado es consistente.

Ejercicio 6

Suponga que cada elemento de una lista de n elementos tiene un valor asociado, y sea $v(i)$ el valor asociado al i -ésimo elemento de la lista. Suponga que n es muy grande, y que además cada elemento puede aparecer en muchos lugares distintos de la lista. Explique cómo se pueden usar números aleatorios para estimar la suma de los valores de los distintos elementos de la lista (donde el valor de cada elemento debe contarse una sola vez, sin importar cuántas veces aparezca en la lista).

Si un elemento k aparece m_k veces, una posición elegida uniformemente lo selecciona con probabilidad m_k/n . Para corregir esa mayor probabilidad causada por las repeticiones, se utiliza el estimador $Z = n \cdot v(k)/m_k$.

$$E[Z] = \sum_k (m_k/n)(n \cdot v(k)/m_k) = \sum_k v(k)$$

Por lo tanto, Z es insesgado ya que su promedio se acerca a la suma de los valores de los elementos distintos. En una lista muy grande se repite el muestreo sobre posiciones aleatorias; si las frecuencias ya están disponibles, se reutilizan para reducir el costo

```
PS C:\Projects\Compiladores-Actividades> py ejercicio6.py
Suma exacta: 135
Suma estimada: 135.4647
Error absoluto: 0.4647
PS C:\Projects\Compiladores-Actividades> 
```

aunque los elementos del ejemplo aparecen con frecuencias diferentes, la corrección por m_k evita contar varias veces su valor y produce una estimación cercana a la suma exacta.

Ejercicio 7

Suponga que $0 \leq \lambda_n \leq \lambda$, para todo $n \geq 1$. Considere el siguiente algoritmo para generar una variable aleatoria con tasas de riesgo (hazard rates) discretas $\{\lambda_n\}$.

PASO 1: $S = 0$.

PASO 2: Genere U y haga $Y = \text{Int} \left(\frac{\log(U)}{\log(1 - \lambda)} \right) + 1$.

PASO 3: $S = S + Y$.

PASO 4: Genere U .

PASO 5: Si $U \leq \lambda_s/\lambda$, haga $X = S$ y deténgase. En caso contrario, regrese al paso 2.

(a) ¿Cuál es la distribución de Y en el Paso 2?

En el paso 2, Y tiene distribución geométrica con parámetro λ y soporte 1, 2, 3, La transformación mostrada en el planteamiento produce:

$$(b) \quad P(Y = n) = (1 - \lambda)^{n-1} \lambda$$

(c) Explique qué está haciendo el algoritmo.

Los valores acumulados S son tiempos candidatos generados con una tasa constante λ . En cada candidato S , el algoritmo acepta con probabilidad λ_s/λ . Este procedimiento se llama adelgazamiento: parte de una tasa que domina a todas las λ_n y elimina candidatos hasta conservar exactamente la tasa deseada.

(d) Argumente que X es una variable aleatoria con tasas de riesgo discretas $\{\lambda_n\}$.

Condicionado a que X todavía no haya ocurrido antes de n , la probabilidad de que n sea candidato es λ . Si es candidato, se acepta con probabilidad λ_n/λ . Por la regla del producto, $P(X = n | X \geq n) = \lambda(\lambda_n/\lambda) = \lambda_n$. Así, X posee las tasas de riesgo solicitadas.

n	λ_n objetivo	Riesgo estimado
1	0.0650	0.0650
2	0.0800	0.0798
3	0.0950	0.0947
4	0.1100	0.1101
5	0.1250	0.1220
6	0.1400	0.1412
7	0.1550	0.1567
8	0.1700	0.1672

```

PS C:\Projects\Compiladores-Actividades> py ejercicio7.py
Simulaciones: 100,000
Promedio de X: 7.4555
n | lambda_n | riesgo estimado
1 | 0.0650   | 0.0650
2 | 0.0800   | 0.0798
3 | 0.0950   | 0.0947
4 | 0.1100   | 0.1101
5 | 0.1250   | 0.1220
6 | 0.1400   | 0.1412
7 | 0.1550   | 0.1567
8 | 0.1700   | 0.1672
PS C:\Projects\Compiladores-Actividades>

```

Los riesgos empíricos obtenidos con 100,000 repeticiones son muy cercanos a las tasas objetivo, lo que respalda la demostración.

Ejercicio 8

Suponga que X y Y son variables aleatorias discretas y que se desea generar el valor de una variable aleatoria W con función de masa de probabilidad

$$P(W = i) = P(X = i \mid Y = j)$$

para algún j especificado tal que $P(Y = j) > 0$. Demuestre que el siguiente algoritmo logra esto.

- Genere el valor de una variable aleatoria con la distribución de X .
- Sea i el valor generado en (a).
- Genere un número aleatorio U .
- Si $U < P(Y = j \mid X = i)$, haga $W = i$ y deténgase.
- Regrese a (a).

En una iteración, el algoritmo propone $X = i$ con probabilidad $P(X = i)$ y luego acepta con probabilidad $P(Y = j \mid X = i)$. Por lo tanto, la probabilidad de aceptar específicamente el valor i es:

$$P(X = i)P(Y = j \mid X = i) = P(X = i, Y = j)$$

La probabilidad total de aceptar cualquier propuesta es la suma sobre i , que equivale a $P(Y = j)$. Condicionado a que hubo aceptación, la probabilidad de que el valor devuelto sea i queda:

$$P(W = i) = P(X = i, Y = j) / P(Y = j) = P(X = i | Y = j)$$

Los rechazos no alteran esta distribución porque cada regreso al inciso (a) inicia un intento independiente con las mismas probabilidades. Solo aumentan el número de intentos necesarios.

```
● PS C:\Projects\Compiladores-Actividades> py ejercicio8.py
Aceptaciones: 100,000
Promedio de intentos por aceptacion: 2.2012
i | simulada | exacta
1 | 0.1753   | 0.1758
2 | 0.3869   | 0.3846
3 | 0.4378   | 0.4396
○ PS C:\Projects\Compiladores-Actividades>
```

Las frecuencias simuladas prácticamente coinciden con las probabilidades condicionales exactas. El promedio fue de 2.2012 intentos por aceptación, cercano a $1/P(Y = j) = 1/0.455 = 2.1978$.

Ejercicio 9

En los incisos 3–9 se utiliza simulación Monte Carlo para aproximar las integrales y se compara cada estimación con el valor exacto o con una referencia numérica de alta precisión. Todas las ejecuciones usan la semilla 23201 y 500,000 muestras por inciso.

Inciso 3

$$\int_0^1 \exp(e^x) dx$$

Si U es uniforme en $[0, 1]$, el intervalo tiene longitud 1 y la integral es $E[\exp(e^U)]$. Se promedian directamente los valores del integrando.

Inciso 4

$$\int_0^1 (1 - x^2)^{3/2} dx$$

Con U uniforme en $[0, 1]$, la integral es $E[(1 - U^2)^{3/2}]$. La respuesta exacta se obtiene por una sustitución trigonométrica.

Inciso 5

$$\int_{-2}^2 e^{(x + x^2)} dx$$

Se transforma U uniforme en $[0, 1]$ mediante $X = -2 + 4U$. Como el intervalo mide 4, el estimador es $4e^{(X+X^2)}$.

Inciso 6

$$\int_0^\infty x(1 + x^2)^{-2} dx$$

Para trabajar en un intervalo finito se usa $X = U/(1-U)$, con jacobiano $1/(1-U)^2$. El promedio incluye el integrando y este jacobiano.

Inciso 7

$$\int_{-\infty}^\infty e^{-x^2} dx$$

La transformación $X = \tan(\pi(U - 1/2))$ cubre toda la recta real. Su jacobiano es $\pi(1+X^2)$, por lo que se promedia $e^{-X^2}\pi(1+X^2)$.

Inciso 8

$$\int_0^1 \int_0^1 e^{-(x+y)^2} dy dx$$

Se generan U y V independientes y uniformes en [0,1]. Como el área del cuadrado es 1, la integral es $E[e^{-(U+V)^2}]$.

Inciso 9

$$\int_0^\infty \int_0^x e^{-(x+y)} dy dx$$

La función $e^{-(x+y)}$ es la densidad conjunta de dos exponenciales independientes de tasa 1. La integral representa $P(Y < X)$, estimada con el indicador 1 si $Y < X$ y 0 en otro caso.

```

● (.venv) PS C:\Projects\Compiladores-Actividades> py ejercicio9.py
Muestras por inciso: 500,000
Inciso | Estimación | Referencia | Error absoluto
-----|-----|-----|-----
3 | 6.314661 | 6.316564 | 0.001903
4 | 0.589275 | 0.589049 | 0.000226
5 | 93.265712 | 93.162753 | 0.102959
6 | 0.500052 | 0.500000 | 0.000052
7 | 1.770760 | 1.772454 | 0.001694
8 | 4.890389 | 4.899159 | 0.008770
9 | 0.500642 | 0.500000 | 0.000642
○ (.venv) PS C:\Projects\Compiladores-Actividades>

```

las siete aproximaciones son cercanas a sus referencias. Los errores no son exactamente cero porque Monte Carlo es un método aleatorio; al aumentar la cantidad de muestras, el error típico disminuye aproximadamente en proporción a $1/\sqrt{n}$. Los incisos 5 y 8 presentan mayor variabilidad porque sus integrandos alcanzan valores grandes dentro del dominio.